Tensor Omics: Data Model Definitions

Asis Hallab, "el Bobito"

April 28, 2025

1 Data Models

All data in Tensor Omics (TOX) must be stored continuously in memory in basic Fortran types. All data must be serializable and deserializable from binary files (`.txdata` format). Those files consist of a header section informing about the contained basic Fortran structures (scalars and arrays), their sizes, and the offset of where to start to read them.

Tensor Omics will require meta-data to be readable and writable from data tables, e.g. `CSV` format. To represent this efficiently we will use the below `DataTable` definitions.

1.1 DataTable

1.1.1 Overview

The **F42 DataTable** implements the **F42 DataStructure** standard. It provides a compact, efficient, and portable representation of structured tabular data (e.g., parsed from CSV files). It is optimized for serialization, patching, and memory-constrained environments.

1.1.2 Public Methods

- `read_from_file`(file, sep_char, column_types, max_char_length, n_rows, column_names)
- `calculate_memory_requirements`()
- `serialize`()
- `deserialize`()
- `log`()
- `update()` (Detailed definition later)
- `save_patch()` (Detailed definition later)

1.1.3 Internal Structure

The DataTable is composed of the following internal components:

- **Type–Mapping Array**: a $2 \times k$ integer array.
  - Row 1: Stores the column type for each input column.
  - Row 2: Stores the index of the corresponding column inside the appropriate data container (see below).
- **Data Containers**:
  - `int_array`: A $n\_rows \times i$ array for integer columns.
  - `real_array`: A $n\_rows \times r$ array for real (floating-point) columns.

- `char_array`: A $n\_rows \times c$ array of fixed-length character sequences (length = max_char_length) for string columns.

where $i + r + c = k$, and each value $i, r, c \geq 0$.

- **Column Names**:
  - A $k \times 256$ character array.
  - Each entry holds the name of a column, truncated or padded to a maximum length of 256 characters.

1.1.4 Detailed Method Definitions

read_from_file

Method signature: `read_from_file(file, sep_char, column_types, max_char_length, n_rows, column_names)`
Reads tabular data from a file and populates the internal structure.

- **file**: Path or handle of the input file.
- **sep_char**: Character used to separate fields (e.g., ‘,‘ or ‘;‘).
- **column_types**: Integer array of length k specifying the type of each input column:
  - 1 = Integer
  - 2 = Real (floating point)
  - 3 = Character (string)
- **max_char_length**: Maximum allowed character length for each string field (in char columns).
- **n_rows**: Expected number of data rows (excluding headers).
- **column_names** (optional): Array of column names. If omitted, defaults to generic names (”col1”, ”col2”, ...).

calculate_memory_requirements

Method signature: `calculate_memory_requirements(n_rows, col_types)`
Estimates and returns the total memory (in bytes) required for the DataTable, including:

- Size of Type–Mapping Array: $2 \times k \times$ sizeof(int)
- Size of Data Containers:
  - $n\_rows \times i \times$ sizeof(int)
  - $n\_rows \times r \times$ sizeof(real)
  - $n\_rows \times c \times$ max_char_length $\times$ sizeof(char)
- Size of Column Names Array: $k \times 256 \times$ sizeof(char)

serialize()

Serializes the DataTable into a binary format consisting of:

1. Header block (generated by `generate_header`).
2. Raw memory blocks for:
   - Type–Mapping Array
   - int_array
   - real_array
   - char_array
   - column_names

All blocks are concatenated sequentially without gaps. Serialization follows little-endian ordering unless otherwise specified.

generate_header()

Creates a header structure containing metadata such as:

- Magic number (identifies the file type, e.g., 'F42DTBL')
- Version number
- Number of rows (n_rows)
- Number of columns (k)
- Maximum character length (max_char_length)
- Column counts per type (i, r, c)

deserialize()

Inverse operation of `serialize()`:

- Reads and verifies the header.
- Reconstructs the Type–Mapping Array and Data Containers from the binary block.
- Ensures compatibility of all dimensions and memory alignment.

1.1.5 Methods to be Specified Later

The following methods will be specified in detail in a future document:

- `log()`
- `update()`
- `save_patch()`

1.1.6 Summary Diagram

$$\text{Input CSV} \xrightarrow{\text{read\_from\_file}} \left\{ \begin{array}{l} \text{Type–Mapping Array} \\ \text{int\_array} \\ \text{real\_array} \\ \text{char\_array} \\ \text{column\_names} \end{array} \right\} \xrightarrow{\text{serialize}} \text{Binary Format}$$